



PRACTICE QUESTIONS FOR DAT605

Case Study 1: PAU Community Food Deliveries (PAUFs)

Background:

PAUFs provides meal delivery services across the Pan-Atlantic University (PAU) campus. To manage its operations, the company relies on a sophisticated database system. As part of its data-driven approach, PAUFs collects and analyses data about customers, orders, meals, delivery times, and ratings. The database is implemented using **PostgreSQL**.

The database (**all tables have a primary key**) has the following schema:

Customers: Information about customers, including Customer ID, Name, Email, and Location.

Orders: Each order placed by a customer, with attributes like OrderID, CustomerID, OrderDate, and TotalAmount.

Meals: Details of meals offered, including MealID, Name, and Category.

OrderDetails: A mapping table between Orders and Meals, with OrderID, MealID, and Quantity.

Ratings: Feedback provided by customers, with RatingID, OrderID, Rating, and Comments. An order may have a rating record (**just one** record if an order was rated at all)

Practice Question 1 [All questions based on the case study]

- (a) Identify the foreign key(s) in the **OrderDetails** table and explain its/their role(s).
- (b) Write the **DDL statement** to create the **OrderDetails** table where the following information is held:
 - i. **OrderID** is a string usually generated by appending the timestamp to the customer ID.
 - ii. **MealID** is an integer
 - iii. **Quantity** is a real value that can store a value with a **two-decimal place** precision.
- (c) Consider the foreign-key constraint you identified from the **OrderDetails** table. Give **two examples** of an **insert statement** to this relation that can cause a violation of the foreign-key constraint. Explain why you provided these insert statements.
- (d) Provide queries to answer the following questions:
 - i. What are the meals that **contain the word 'Chicken'** in them? A customer just wants a meal that has chicken in it.
 - ii. Provide the name of the **most frequently** ordered meal.
 - iii. What is the **total revenue** generated from the customer 'Jade Kosoko'? Explain how you arrived at the query you provided.

Practice Question 2 [Question 2b not necessarily based on Case Study 1]

- (a) What are **weak entities** in database modelling? Give an example (with explanation) of a weak entity for any of the relations in PAUFs.
- (b) Explain the following terms with **an example for each**:
 - i. Specialisation
 - ii. Multi-valued Attributes
 - iii. Partial Participation
- (c) If the meals table needs new information to be captured (**unit_price**, a numeric value of **two-decimal place** precision), answer the following questions:
 - i. Alter the meals table to reflect the new column (**unit_price**).
 - ii. Update the meals table to set the unit price of 'Grilled Chicken Salad' to 4500.
 - iii. Insert a new meal (provide arbitrary details for the meal) into the meals table.

[Note: Insert based on the expected change on the table as per Q2c(i)]
- (d) Use the **customers** and **orders** table to differentiate between a **LEFT JOIN** and a **RIGHT JOIN**. Support the explanation of the two concepts with the expected results of your sample queries.

Practice Question 3

Case Study 2: MediCare Clinic is designing a new patient management system. They have identified Patient as a strong entity with attributes PatientID, Name, Address, and DateOfBirth. They also have an Appointment as an entity that is **weakly dependent** on Patient, with attributes AppointmentID (partial identifier), AppointmentDate, and Reason. Furthermore, a Patient can have multiple Doctors, and a Doctor can have multiple Patients. Doctors have DoctorID, Name, and Speciality (a doctor must have only one speciality). However, there is a **ConsultationDate** attribute that may be taken between a patient and a doctor **only if** a patient consults a doctor.

- (a) Write a set of **DDL create table** commands for the tables (one table creation command per table) that would result from your analysis of their short business rule.
- (b) Provide a **valid insert** statement that would populate the appropriate table with three sample doctor records.
- (c) Why is it necessary (after creating the tables in the database) to push data into the patient table before the appointment table? Let your answer be as clear as possible.

Practice Question 4

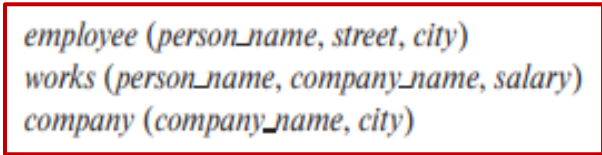
- (a) Explain the following terms with examples:
 - i. Data Manipulation Language (DML).
 - ii. Data Definition Language (DDL).
 - iii. Embedded SQL vs. Dynamic SQL.
- (b) Explain the components of a database system.

- (c) As a database consultant, please provide answers with the relevant SQL commands to some of the questions regarding the **PAUFs Case Study**:
- Find the customerids of customers whose orders were rated between 3 and 4 **without repeating** any customerid in the result table.
 - The price attribute of the Meals table was created as an integer; modify its property such that it can handle real numbers of seven digits with a numeric precision of 2.
 - There have been some data integrity issues with the values inputted into the **OrderedMeals** table lately. Introduce a constraint to the table, making sure the **quantity** value of a record will no longer be negative.

Practice Question 5

Consider the employee database in Figure 1. Write SQL statements to answer the following questions:

- Find the name of each employee who lives in the city: “Abuja”.
- Find the name of each employee whose salary is greater than ₦100000.
- Find the name of each employee who lives in “Ibadan” and whose salary is greater than ₦100000.



```
employee (person_name, street, city)
works (person_name, company_name, salary)
company (company_name, city)
```

Figure 1: Employee database

- Discuss **three (3)** key advantages of using a Database Management System (DBMS) like PostgreSQL over traditional file systems for setting up a company’s database, and mention **three (3)** alternatives that could have been used instead of PostgreSQL.

Practice Question 6

Pan-Atlantic University (PAU) is developing a new database system to manage its diverse events. Here are the key requirements:

- Each Event (e.g., "Annual Gala", "Sports Fest") is uniquely identified by an EventID. It also has a Name, Description, EventDate, and Location. The Name, EventDate, and Location of an event **must always be provided**.
- Attendees (AttendeeID, Name, Email, PhoneNumber) register for events. Each attendee **must have a unique email address**, and **multiple phone numbers** can be provided.
- An event can have any number of attendees, and an attendee can register for multiple events (one or more). When an attendee registers for an event, the RegistrationDate and a unique ConfirmationCode (for that specific registration instance) are recorded.
- Speakers (SpeakerID, Name, Bio, ContactEmail) are invited to speak at events. An event must have a speaker, and can feature multiple speakers, and a speaker can be invited to

multiple events. For each speaker's involvement in a particular event, the PresentationTopic and PresentationTime are noted.

- Venues (VenueID, VenueName, Capacity, Address) host events. An event is hosted at exactly one venue, and an event must always be associated with a venue. A venue can host multiple events over time, and a venue does not necessarily have to host any event.

Answer the following questions:

- (a) Draw an Entity-Relationship (ER) Diagram for the University Event Management System. **Ensure you represent:**
 - All entities with their attributes (including identifiers and multi-valued attributes).
 - All relationships with their cardinality constraints.
- (b) Map the **ER Diagram** to a set of Relational Schemas (Database Tables). For each schema, mark the primary keys by underlining them and **also state/note** the foreign keys (for the tables) if applicable. The foreign key names **MUST MATCH** the primary key names of the **referenced table**. **Follow the format below**, which presents a random schema with a composite key and a foreign key.
 - classroom(roomNumber, building, capacity); building is a foreign key
- (c) Write **SQL commands/queries** for the following tasks, assuming the tables you defined in (b) are created and populated with some sample data:
 - i. Create the Events table, ensuring you pay attention to the details of the **Event** table attributes in the case study provided.
 - ii. Find the **names** of all attendees who registered for the event with **EventID 'E001'**.
- (d) List all venues and the total number of events they have hosted. Include venues that have not hosted any events, **showing a count of 0 for such**. Order the result by the number of events in descending order.